

Projeto: Desenvolvimento Back-end

CST ANÁLISE E DESENVOLVIMENTO DE SISTEMAS – DISTÂNCIA

**API REST para Gestão de Pedidos – Rede Raízes do
Nordeste**

Aluno: Guilherme Faria de Paula

SUMÁRIO

1 INTRODUÇÃO E OBJETIVOS

2 ANÁLISE E REQUISITOS

2.1 Requisitos Funcionais

2.2 Requisitos Não Funcionais

2.3 Priorização e Justificativa Técnica do MVP

3 MODELAGEM E DIAGRAMAS

3.1 Diagrama de Casos de Uso

3.2 Descrição de Feature — Realizar Pedido + Solicitar Pagamento

3.3 DER — Diagrama Entidade-Relacionamento

3.4 Diagrama de Classes

3.5 Diagrama de Sequência — Fluxo Crítico

4 ARQUITETURA DA SOLUÇÃO

4.1 Stack Tecnológica

4.2 Estrutura do Repositório

4.3 Organização em Camadas

5 API E ENDPOINTS

5.1 Padrão de Erro

5.2 Autenticação

5.3 Unidades

5.4 Produtos

5.5 Estoque

5.6 Pedidos

5.7 Fidelidade

5.8 Auditoria

6 LGPD, PRIVACIDADE E SEGURANÇA

7 ENTREGA TÉCNICA

8 PLANO DE TESTES

9 CONCLUSÃO

1 INTRODUÇÃO E OBJETIVOS

O presente trabalho apresenta o desenvolvimento de uma API REST para a gestão de pedidos da rede fictícia Raízes do Nordeste, no contexto da disciplina de Projeto Multidisciplinar da trilha Back-End do curso de Tecnologia em Análise e Desenvolvimento de Sistemas.

A solução foi construída utilizando Python e FastAPI, com persistência de dados em SQLite por meio do SQLAlchemy ORM. A API implementa autenticação baseada em JWT, controle de acesso por perfil de usuário, gerenciamento de unidades, produtos, estoque, pedidos, pagamento mock, programa de fidelidade e registro de auditoria.

O objetivo principal do projeto é demonstrar a construção de um MVP funcional, testável e organizado, capaz de representar um fluxo realista de pedidos em uma rede de restaurantes. O sistema permite que usuários autenticados criem pedidos, tenham seus pagamentos processados por um serviço mock determinístico, atualizem estoque, acumulem pontos de fidelidade e gerem registros de auditoria para ações relevantes.

A proposta é entregar uma base técnica consistente, adequada para fins acadêmicos, com separação de responsabilidades, persistência de dados, autenticação, autorização, regras de negócio e documentação automática da API. A API foi projetada para atender aos seguintes objetivos técnicos:

- implementar endpoints REST organizados por domínio funcional;
- autenticar usuários por meio de JWT;
- aplicar autorização por perfil de acesso;
- persistir dados com SQLAlchemy ORM;
- validar dados de entrada com Pydantic;
- documentar automaticamente a API via Swagger/OpenAPI;
- simular pagamento externo de forma previsível;
- manter rastreabilidade por meio de logs de auditoria;
- aplicar soft-delete em recursos principais;
- estruturar o projeto de forma modular e organizada;
- disponibilizar coleção Postman para validação reproduzível.

2 ANÁLISE E REQUISITOS

A análise do sistema foi realizada a partir do estudo de caso da rede Raízes do Nordeste, considerando uma operação de restaurante com múltiplas unidades e diferentes canais de atendimento. O back-end precisa centralizar as regras de negócio relacionadas a usuários, produtos, unidades, estoque, pedidos, pagamento, fidelidade e auditoria. A API também precisa garantir controle de acesso, armazenamento seguro de senhas, rastreabilidade de ações sensíveis e documentação clara para testes e validação.

2.1 Requisitos Funcionais

ID	Requisito	Status
RF01	Permitir cadastro de usuário com nome, e-mail, senha, perfil e consentimento LGPD	Implementado
RF02	Permitir login com e-mail e senha, retornando token JWT	Implementado
RF03	Permitir listagem e consulta de unidades ativas	Implementado
RF04	Permitir cadastro, atualização e desativação de unidades	Implementado
RF05	Permitir listagem e consulta de produtos ativos	Implementado
RF06	Permitir cadastro, atualização e desativação de produtos	Implementado
RF07	Permitir consulta de estoque por unidade	Implementado
RF08	Permitir registro de entrada de estoque	Implementado
RF09	Permitir consulta de movimentações de estoque	Implementado
RF10	Permitir criação de pedido com unidade, canal, forma de pagamento e itens	Implementado
RF11	Validar disponibilidade de estoque antes da criação do pedido	Implementado
RF12	Processar pagamento mock automaticamente durante a criação do pedido	Implementado
RF13	Atualizar status do pedido conforme resultado do pagamento	Implementado
RF14	Baixar estoque em pedidos aprovados	Implementado
RF15	Acumular pontos de fidelidade em pedidos aprovados	Implementado
RF16	Permitir consulta de saldo de fidelidade pelo cliente	Implementado
RF17	Permitir consulta de saldo de fidelidade por administradores e gerentes	Implementado
RF18	Permitir resgate de pontos	Implementado
RF19	Registrar ações relevantes em log de auditoria	Implementado

ID	Requisito	Status
RF20	Permitir consulta dos registros de auditoria com filtros	Implementado
RF21	Permitir atualização manual do status do pedido pela cozinha	Implementado
RF22	Permitir promoções/campanhas com desconto no pedido	Proposta

Tabela 1 — Requisitos Funcionais

2.2 Requisitos Não Funcionais

ID	Requisito
RNF01	A API deve utilizar arquitetura modular, separando rotas, serviços e infraestrutura
RNF02	A autenticação deve ser baseada em JWT
RNF03	As senhas devem ser armazenadas com hash seguro
RNF04	A autorização deve considerar perfis de usuário
RNF05	A API deve possuir documentação automática via Swagger/OpenAPI
RNF06	As respostas de erro devem seguir padrão JSON estruturado
RNF07	O banco de dados deve ser acessado por ORM
RNF08	O sistema deve registrar logs de auditoria para ações importantes
RNF09	O pagamento mock deve ter comportamento determinístico para facilitar testes
RNF10	A estrutura deve ser organizada e permitir manutenção do código
RNF11	O sistema deve permitir validação reproduzível por coleção Postman
RNF12	O sistema deve preservar dados relevantes por soft-delete em recursos principais

Tabela 2 — Requisitos Não Funcionais

2.3 Priorização e Justificativa Técnica do MVP

A priorização do MVP seguiu o critério de entregar primeiro o fluxo operacional mais crítico para a rede Raízes do Nordeste: criação de pedido, validação de itens, processamento de pagamento mock, atualização automática de status, baixa de estoque, acúmulo de pontos de fidelidade e registro de auditoria.

O fluxo principal implementado foi o fluxo Pedido → Pagamento Mock → Atualização de Status. Esse fluxo foi priorizado porque representa o núcleo da operação de venda: sem pedido, pagamento e mudança de status, o sistema não demonstra funcionamento real de back-end.

A multicanalidade também foi tratada como requisito de domínio. O pedido possui o campo `canal_pedido`, permitindo registrar a origem do pedido, como APP,

TOTEM, BALCAO, PICKUP ou WEB. A API também permite filtrar pedidos por canal, garantindo rastreabilidade operacional entre os diferentes canais de atendimento.

Alguns requisitos foram documentados como proposta para evolução, sem serem tratados como implementados no código atual. É o caso das promoções/campanhas. A atualização manual do status do pedido pela cozinha, no fluxo EM_PREPARO → PRONTO → ENTREGUE/CANCELADO, foi implementada via endpoint PATCH /pedidos/{pedido_id}/status, com controle de acesso restrito a ADMIN e GERENTE e registro de auditoria.

Dessa forma, o projeto busca equilíbrio entre escopo acadêmico, funcionamento real e coerência técnica: o que está implementado foi validado por API e coleção Postman; o que ficou como proposta foi identificado claramente como evolução documentada.

3 MODELAGEM E DIAGRAMAS

Esta seção apresenta a modelagem da solução proposta para a rede Raízes do Nordeste. A modelagem foi construída com foco no fluxo principal do sistema: criação de pedido, validação de estoque, processamento de pagamento mock, atualização de status, acúmulo de pontos de fidelidade e registro de auditoria. Os diagramas e descrições a seguir representam a visão de domínio, os principais atores, as entidades persistidas no banco de dados e o fluxo crítico implementado na API.

3.1 Diagrama de Casos de Uso

O sistema possui cinco grupos principais de atores: Cliente, Atendente, Cozinha, Gerente/Administrador e Gateway de Pagamento Mock. A Figura 1 apresenta o diagrama de casos de uso da solução.

3.1 Diagrama de Casos de Uso

O sistema possui cinco grupos principais de atores: Cliente, Atendente, Cozinha, Gerente/Administrador e Gateway de Pagamento Mock. A Figura 1 apresenta o diagrama de casos de uso da solução.

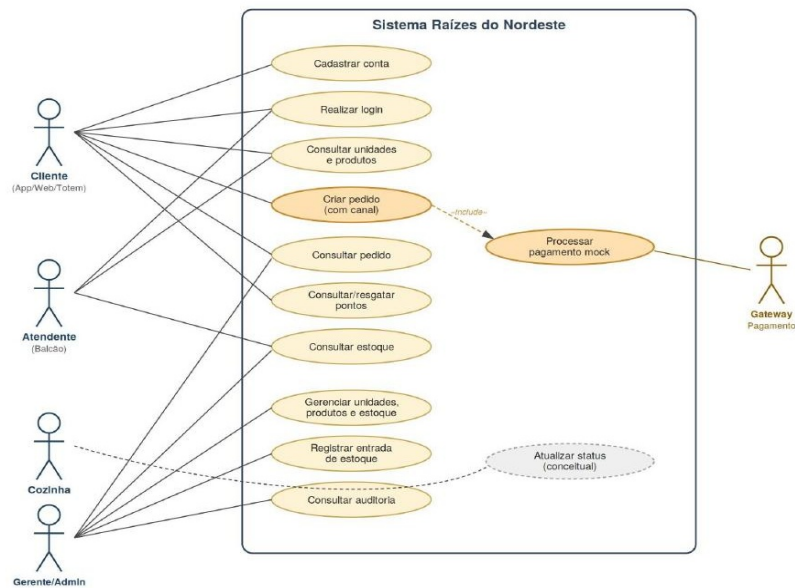


Figura 1 — Diagrama de Casos de Uso

Figura 1 — Diagrama de Casos de Uso

Ator	Casos de Uso
Cliente	Cadastrar conta, realizar login, consultar unidades, consultar produtos, criar pedido, consultar pedido, consultar saldo de fidelidade, resgatar pontos

Ator	Casos de Uso
Atendente	Realizar login, consultar produtos, consultar estoque, apoiar criação de pedidos de balcão
Cozinha	Acompanhar pedidos em preparo, atualizar status do pedido para pronto ou entregue, cancelar pedido quando necessário
Gerente/Administrador	Gerenciar unidades, produtos, estoque, movimentações, auditoria e pedidos por canal
Gateway de Pagamento Mock	Receber solicitação de pagamento e retornar pagamento aprovado ou recusado

Tabela 3 — Casos de Uso por Ator

No MVP implementado, o fluxo executável concentra-se no cliente criando pedidos, no pagamento mock determinístico e na atualização automática do status conforme aprovação ou recusa. A atualização manual de status pela cozinha foi implementada via endpoint PATCH /pedidos/{pedido_id}/status.

3.2 Descrição de Feature — Realizar Pedido + Solicitar Pagamento

Campo	Descrição
Identificador	UC-PED-01
Nome	Realizar Pedido e Solicitar Pagamento
Objetivo	Permitir que um cliente autenticado crie um pedido, tenha o estoque validado e o pagamento processado por serviço mock
Ator principal	Cliente
Atores secundários	Gateway de Pagamento Mock, Estoque, Fidelidade, Auditoria
Endpoint principal	POST /pedidos/
Pré-condições	Usuário autenticado com JWT válido; unidade existente; produtos existentes; estoque suficiente; canal do pedido informado
Pós-condições de sucesso	Pedido criado; pagamento aprovado; status atualizado para EM_PREPARO; estoque baixado; pontos acumulados; auditoria registrada
Pós-condições de falha	Pedido marcado como PAGAMENTO_RECUSADO; estoque não é baixado; pontos não são acumulados

3.3 Fluxo principal:

1. O cliente realiza login e obtém um token JWT.
2. O cliente envia POST /pedidos/ com unidade, canal, forma de pagamento e itens.

3. A API valida o token JWT.
4. A API valida os dados enviados no corpo da requisição.
5. O sistema verifica se a unidade existe.
6. O sistema verifica se os produtos existem.
7. O sistema verifica se há estoque suficiente.
8. O sistema calcula o total do pedido.
9. O sistema cria o pedido com status inicial.
10. O serviço de pagamento mock é acionado.
11. Se o pagamento for aprovado, o status passa para EM_PREPARO.
12. O estoque dos itens é baixado.
13. Os pontos de fidelidade são acumulados.
14. A ação é registrada em auditoria.
15. A API retorna HTTP 201 com os dados do pedido e do pagamento.

Fluxos alternativos e exceções:

- Pagamento recusado: quando o serviço mock retorna recusa, o pedido recebe status PAGAMENTO_RECUSADO, o pagamento é registrado como RECUSADO, o estoque não é baixado e os pontos não são acumulados.
- Produto inexistente: a API retorna erro 404 com mensagem padronizada.
- Acesso sem autenticação: a API bloqueia a requisição e retorna erro 401

Regras de negócio:

Regra	Descrição
RN01	Todo pedido deve possuir canal de origem
RN02	O canal do pedido deve permitir rastreabilidade operacional
RN03	O pedido só pode ser criado com produtos existentes
RN04	O sistema deve validar estoque antes de concluir o fluxo
RN05	Pagamento aprovado atualiza o pedido para EM_PREPARO
RN06	Pagamento recusado atualiza o pedido para PAGAMENTO_RECUSADO
RN07	Estoque só é baixado quando o pagamento é aprovado
RN08	Pontos de fidelidade só são acumulados quando o pagamento é aprovado
RN09	Ações relevantes devem ser registradas em auditoria

Tabela 4 — Regras de Negócio do Fluxo Crítico

3.3 DER — Diagrama Entidade-Relacionamento

O banco de dados foi modelado com SQLAlchemy ORM e utiliza SQLite no ambiente do projeto. As entidades representam usuários, unidades, produtos, estoque, pedidos, pagamentos, fidelidade e auditoria. A Figura 2 apresenta o DER da solução.

Tabela 4 — Regras de Negócio do Fluxo Crítico

3.3 DER — Diagrama Entidade-Relacionamento

O banco de dados foi modelado com SQLAlchemy ORM e utiliza SQLite no ambiente do projeto. As entidades representam usuários, unidades, produtos, estoque, pedidos, pagamentos, fidelidade e auditoria. A Figura 2 apresenta o DER da solução.

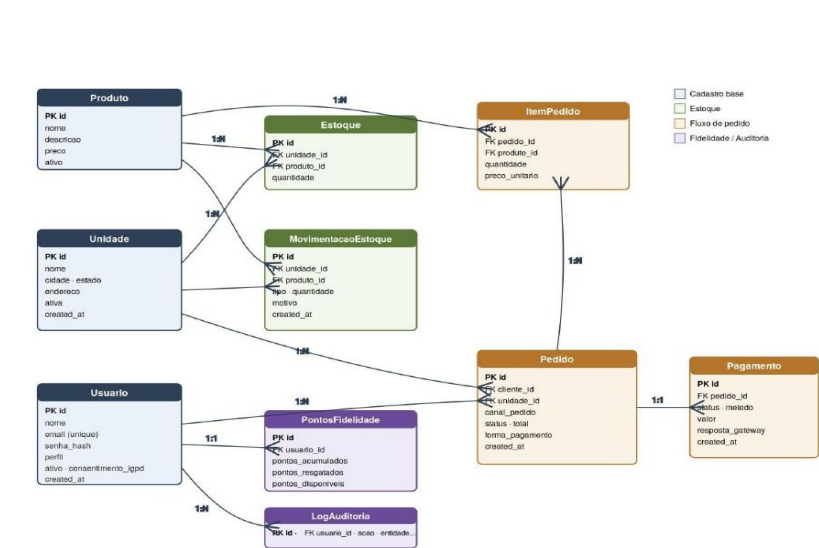


Figura 2 — Diagrama Entidade-Relacionamento (DER)

Figura 2 — Diagrama Entidade-Relacionamento (DER)

Entidade	Atributos principais	Relacionamentos
Usuario	id, nome, email, senha_hash, perfil, ativo, consentimento_lgpd, created_at	Vários pedidos, um saldo de fidelidade e vários registros de auditoria
Unidade	id, nome, cidade, estado, endereco, ativa, created_at	Vários estoques, movimentações e pedidos
Produto	id, nome, descricao, preco, ativo	Estoques, movimentações e itens de pedido
Estoque	id, unidade_id, produto_id, quantidade	Relaciona unidade e produto
MovimentacaoEstoque	id, unidade_id, produto_id, tipo, quantidade, motivo, created_at	Registra entradas e saídas
Pedido	id, cliente_id, unidade_id, canal_pedido, status, total, forma_pagamento, created_at	Usuário, unidade, itens e pagamento

Entidade	Atributos principais	Relacionamentos
ItemPedido	id, pedido_id, produto_id, quantidade, preco_unitario	Produtos incluídos em um pedido
Pagamento	id, pedido_id, status, metodo, valor, resposta_gateway, created_at	Resultado do pagamento mock
PontosFidelidade	id, usuario_id, pontos_acumulados, pontos_resgatados, pontos_disponiveis	Saldo de pontos do cliente
LogAuditoria	id, usuario_id, acao, entidade, entidade_id, detalhes, created_at	Ações sensíveis realizadas

Tabela 5 — Entidades e Relacionamentos

Cardinalidades principais: Usuario 1:N Pedido; Usuario 1:1 PontosFidelidade; Unidade 1:N Estoque; Produto 1:N Estoque; Pedido 1:N ItemPedido; Pedido 1:1 Pagamento; Usuario 1:N LogAuditoria.

Restrições relevantes: e-mail único; estoque controlado por unidade e produto; pedido com ao menos um item; canal obrigatório; pagamento vinculado a pedido; soft-delete em unidades e produtos; movimentações de estoque para rastrear entradas e saídas.

3.4 Diagrama de Classes

O diagrama de classes representa a visão de domínio do sistema, com os principais objetos, seus atributos e relacionamentos. A Figura 3 apresenta as classes da solução.

3.4 Diagrama de Classes

O diagrama de classes representa a visão de domínio do sistema, com os principais objetos, seus atributos e relacionamentos. A Figura 3 apresenta as classes da solução.

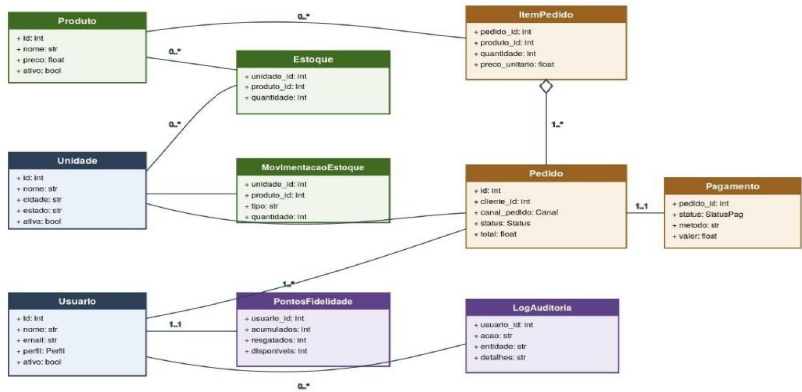


Figura 3 — Diagrama de Classes (visão de domínio)

3.5 Diagrama de Sequência — Fluxo Crítico

O diagrama de sequência detalha o fluxo crítico Pedido → Pagamento Mock → Atualização de Status, mostrando a troca de mensagens entre os componentes. A Figura 4 apresenta esse

Figura 3 — Diagrama de Classes (visão de domínio)

3.5 Diagrama de Sequência — Fluxo Crítico

O diagrama de sequência detalha o fluxo crítico Pedido → Pagamento Mock → Atualização de Status, mostrando a troca de mensagens entre os componentes. A Figura 4 apresenta esse fluxo, incluindo o caminho alternativo de pagamento recusado.

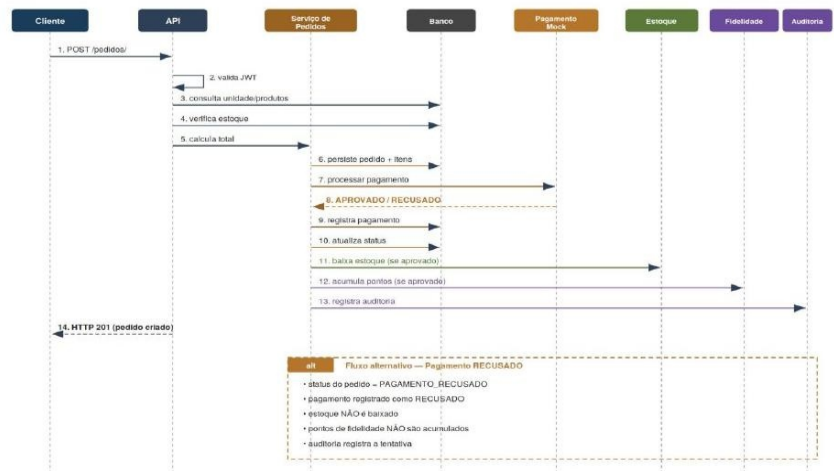


Figura 4 — Diagrama de Sequência: Pedido, Pagamento Mock e Atualização de Status

Passo	Origem	Destino	Ação
1	Cliente	API	Envia POST /pedidos/ com unidade, canal, forma de pagamento e itens
2	API	API	Valida token JWT e permissões
3	API	Banco de Dados	Consulta unidade e produtos
4	API	Banco de Dados	Verifica estoque disponível
5	API	Serviço de Pedidos	Calcula total do pedido
6	Serviço de Pedidos	Banco de Dados	Persiste pedido e itens
7	Serviço de Pedidos	Pagamento Mock	Solicita processamento do pagamento
8	Pagamento Mock	Serviço de Pedidos	Retorna pagamento aprovado ou recusado
9	Serviço de Pedidos	Banco de Dados	Registra pagamento
10	Serviço de Pedidos	Banco de Dados	Atualiza status do pedido
11	Serviço de Estoque	Banco de Dados	Baixa estoque se aprovado
12	Serviço de Fidelidade	Banco de Dados	Acumula pontos se aprovado
13	Serviço de Auditoria	Banco de Dados	Registra ação sensível
14	API	Cliente	Retorna HTTP 201

Tabela 6 — Passos do Fluxo Crítico

4 ARQUITETURA DA SOLUÇÃO

4.1 Stack Tecnológica

Tecnologia	Função
Python	Linguagem principal
FastAPI	Framework web para construção da API
Uvicorn	Servidor ASGI para execução
SQLAlchemy	ORM para mapeamento objeto-relacional
SQLite	Banco de dados utilizado no projeto
Pydantic	Validação e serialização de dados
Passlib	Hashing de senhas
bcrypt	Algoritmo de hashing utilizado pelo Passlib
python-jose	Geração e validação de tokens JWT
python-dotenv	Carregamento de variáveis de ambiente
python-multipart	Suporte a login via form-data
email-validator	Validação de campos de e-mail usados pelo Pydantic

Tabela 7 — Stack Tecnológica

4.2 Estrutura do Repositório

A estrutura principal do projeto é organizada da seguinte forma:

```
raizes_nordeste/
|-- main.py
|-- seed.py
|-- requirements.txt
|-- README.md
|-- .env.example
|-- raizes_nordeste_postman.json
`-- app/
    |-- api/
    | |-- auth.py
    | |-- unidades.py
    | |-- produtos.py
```

```

| |-- estoque.py
| |-- pedidos.py
| |-- fidelidade.py
| |-- auditoria.py
| `-- deps.py
|-- application/
| |-- auditoria_service.py
| |-- fidelidade_service.py
| `-- pagamento_service.py
|-- infrastructure/
| |-- database.py
| |-- models.py
| `-- security.py
`-- domain/
    `-- __init__.py

```

4.3 Organização em Camadas

Camada	Diretório	Responsabilidade
API	app/api/	Define endpoints HTTP, valida entrada, aplica autenticação e autorização
Application	app/application/	Contém serviços de negócio, como pagamento, fidelidade e auditoria
Infrastructure	app/infrastructure/	Contém banco de dados, modelos ORM e funções de segurança
Domain	app/domain/	Reservado para organização e evolução de regras de domínio

Tabela 8 — Organização em Camadas

Essa organização facilita manutenção e expansão, pois separa a camada HTTP da persistência e dos serviços internos.

5 API E ENDPOINTS

A API segue o padrão REST, utilizando recursos no plural, métodos HTTP coerentes e autenticação JWT em endpoints protegidos.

5.1 Padrão de Erro

As respostas de erro seguem estrutura JSON padronizada, contendo nome do erro, mensagem legível, detalhes, timestamp e rota.

```
{
  "error": "NOME_DO_ERRO",
  "message": "Mensagem legível para o usuário",
  "details": [],
  "timestamp": "2026-06-05T12:00:00",
  "path": "/rota"
}
```

5.2 Autenticação

Método	Endpoint	Acesso	Descrição
POST	/auth/registro	Público	Cadastra novo usuário
POST	/auth/login	Público	Autentica usuário e retorna token JWT

Tabela 9 — Endpoints de Autenticação

```
curl -X POST http://127.0.0.1:8000/auth/login \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "username=admin@raizes.com&password;=Admin@123"

{
  "access_token": "TOKEN_JWT",
  "token_type": "bearer",
  "perfil": "ADMIN",
  "nome": "Admin"
}
```

5.3 Unidades

Método	Endpoint	Acesso	Descrição
GET	/unidades/	Público	Lista unidades ativas

Método	Endpoint	Acesso	Descrição
GET	/unidades/{unidade_id}	Público	Consulta unidade ativa por ID
POST	/unidades/	ADMIN / GERENTE	Cria nova unidade
PUT	/unidades/{unidade_id}	ADMIN / GERENTE	Atualiza unidade
DELETE	/unidades/{unidade_id}	ADMIN	Realiza soft-delete da unidade

Tabela 10 — Endpoints de Unidades

O delete de unidade não remove fisicamente o registro do banco. Ele altera o campo ativa para False.

5.4 Produtos

Método	Endpoint	Acesso	Descrição
GET	/produtos/	Público	Lista produtos ativos
GET	/produtos/{produto_id}	Público	Consulta produto por ID
POST	/produtos/	ADMIN / GERENTE	Cria produto
PUT	/produtos/{produto_id}	ADMIN / GERENTE	Atualiza produto
DELETE	/produtos/{produto_id}	ADMIN / GERENTE	Realiza soft-delete do produto

Tabela 11 — Endpoints de Produtos

O delete de produto altera o campo ativo para False, preservando histórico e integridade referencial.

5.5 Estoque

Método	Endpoint	Acesso	Descrição
GET	/estoque/{unidade_id}	ADMIN / GERENTE	Consulta estoque de uma unidade
POST	/estoque/entrada	ADMIN / GERENTE	Registra entrada de estoque
GET	/estoque/{unidade_id}/movimentacoes	ADMIN / GERENTE	Lista movimentações da unidade

Tabela 12 — Endpoints de Estoque

5.6 Pedidos

Método	Endpoint	Acesso	Descrição
POST	/pedidos/	CLIENTE	Cria pedido e processa pagamento mock
GET	/pedidos/	ADMIN / GERENTE / CLIENTE	Lista pedidos conforme perfil
GET	/pedidos/{pedido_id}	ADMIN / GERENTE / dono	Consulta detalhes de um pedido
PATCH	/pedidos/{pedido_id}/status	ADMIN / GERENTE	Atualiza status do pedido (fluxo de cozinha)

Tabela 13 — Endpoints de Pedidos

A listagem de pedidos permite filtro por canal: /pedidos/?canal_pedido=APP

O endpoint PATCH /pedidos/{pedido_id}/status permite que ADMIN e GERENTE atualizem manualmente o status de um pedido seguindo as transições válidas: EM_PREPARO → PRONTO, EM_PREPARO → CANCELADO e PRONTO → ENTREGUE. Transições inválidas retornam HTTP 422. A ação é registrada em auditoria.

Exemplo – pedido aprovado:

```
{ "unidade_id": 1, "canal_pedido": "APP", "forma_pagamento": "PIX",
  "itens": [{"produto_id": 1, "quantidade": 1}] }
```

Exemplo – pedido recusado:

```
{ "unidade_id": 1, "canal_pedido": "APP", "forma_pagamento":
  "MOCK_RECUSADO",
  "itens": [{"produto_id": 1, "quantidade": 1}] }
```

Exemplo – atualização de status (cozinha):

```
PATCH /pedidos/1/status
{ "novo_status": "PRONTO" }
```

5.7 Fidelidade

Método	Endpoint	Acesso	Descrição
GET	/fidelidade/saldo	CLIENTE	Consulta saldo do usuário autenticado
GET	/fidelidade/saldo/{usuario_id}	ADMIN / GERENTE	Consulta saldo de outro usuário
POST	/fidelidade/resgatar	CLIENTE	Resgata pontos de fidelidade

Tabela 14 — Endpoints de Fidelidade

5.8 Auditoria

Método	Endpoint	Acesso	Descrição
GET	/auditoria/	ADMIN	Lista registros de auditoria com filtros opcionais

Tabela 15 — Endpoints de Auditoria

6 LGPD, PRIVACIDADE E SEGURANÇA

A API implementa mecanismos básicos de segurança adequados ao escopo do projeto.

6.1 Dados Pessoais e Finalidade

Dado	Finalidade
Nome	Identificação do usuário no sistema
E-mail	Login e identificação única
Senha em hash	Autenticação segura
Pedidos	Operação do serviço e histórico
Pontos de fidelidade	Programa de benefícios
Logs de auditoria	Rastreabilidade de ações sensíveis

Tabela 16 — Dados Pessoais e Finalidade

6.2 Hashing de Senhas

As senhas não são armazenadas em texto puro. O sistema utiliza Passlib com bcrypt para gerar hashes seguros. Antes de aplicar bcrypt, a senha passa por preparação interna com SHA-256 e Base64, evitando limitações de tamanho do bcrypt e padronizando a entrada.

6.3 Autenticação JWT

O login retorna um token JWT contendo informações essenciais do usuário, como ID e perfil. Esse token deve ser enviado no header Authorization das requisições protegidas, no formato Authorization: Bearer TOKEN_JWT.

6.4 Autorização por Perfil

O sistema possui controle de acesso por perfil. Os principais perfis são ADMIN, GERENTE e CLIENTE. Cada endpoint protegido exige um ou mais perfis autorizados.

6.5 Consentimento LGPD

O cadastro exige aceite de consentimento LGPD. Esse consentimento representa ciência do usuário sobre o tratamento dos dados dentro do sistema. Além disso, o sistema utiliza soft-delete em recursos principais, preservando rastreabilidade e evitando remoções físicas imediatas de dados importantes para histórico operacional.

6.6 Auditoria

O log de auditoria permite rastrear ações relevantes no sistema. Isso melhora a transparência operacional e facilita análise de eventos sensíveis.

7 ENTREGA TÉCNICA

A entrega técnica do projeto foi organizada para permitir que o avaliador consiga acessar, executar e validar a API localmente. O repositório público contém o código-fonte, a estrutura de pastas, o arquivo de dependências, o script de seed, a coleção Postman e as instruções necessárias para execução.

7.1 Repositório Público

O código-fonte do projeto está disponível em repositório público no GitHub:

<https://github.com/Guilherme-faria21/raizes-nordeste-api>

- código-fonte do back-end;
- estrutura organizada em camadas;
- arquivo requirements.txt com dependências;
- arquivo .env.example para configuração;
- script seed.py para criação de dados iniciais;
- arquivo main.py para inicialização da aplicação;
- coleção Postman raizes_nordeste_postman.json;
- README com instruções de execução;

7.2 Execução Local da API

```
git clone https://github.com/Guilherme-faria21/raizes-nordeste-api.git
cd raizes-nordeste-api

python3 -m venv .venv

source .venv/bin/activate

pip install -r requirements.txt

python seed.py

uvicorn main:app --reload
```

Após a inicialização, a API fica disponível localmente em <http://127.0.0.1:8000>.

7.3 Documentação Swagger/OpenAPI

Swagger UI: <http://127.0.0.1:8000/docs>

OpenAPI JSON: <http://127.0.0.1:8000/openapi.json>

7.4 Coleção Postman

O projeto inclui a coleção Postman raizes_nordeste_postman.json. Ela contém testes para autenticação, pedidos, pagamento mock, fidelidade, auditoria, unidades e cenários de erro.

Variável	Finalidade
base_url	URL base da API local
token_admin	Token JWT do usuário administrador
token_cliente	Token JWT do usuário cliente
pedido_id	ID do pedido aprovado criado durante os testes
pedido_recusado_id	ID do pedido recusado criado durante os testes
unidade_teste_id	ID da unidade criada para teste de soft-delete

Tabela 17 — Variáveis da Coleção Postman

7.5 Credenciais de Teste

Perfil	E-mail	Senha
ADMIN	admin@raizes.com	Admin@123
GERENTE	gerente@raizes.com	Gerente@123
CLIENTE	maria@email.com	Cliente@123
CLIENTE	joao@email.com	Cliente@123

Tabela 18 — Credenciais de Teste (seed)

8 PLANO DE TESTES

A validação da API foi realizada por meio da coleção Postman `raizes_nordeste_postman.json`, disponível no repositório do projeto. A execução final da coleção resultou em 52/52 asserções aprovadas. Os testes foram organizados para cobrir cenários positivos e negativos dos principais módulos da API, incluindo autenticação, autorização, pedidos, pagamento mock, fidelidade, auditoria, unidades e regras de negócio.

8.1 Resumo da Validação

Item	Resultado
Ferramenta principal	Postman
Arquivo	<code>raizes_nordeste_postman.json</code>
Total de asserções	52
Asserções aprovadas	52
Resultado final	52/52 PASS
API local	<code>http://127.0.0.1:8000</code>
Documentação	Swagger/OpenAPI em <code>/docs</code>

Tabela 19 — Resumo da Validação

8.2 Cenários de Teste

ID	Cenário	Endpoint	Entrada	Saída esperada
T01	Login administrador válido	POST <code>/auth/login</code>	E-mail e senha do ADMIN	HTTP 200, token JWT, perfil ADMIN
T02	Login cliente válido	POST <code>/auth/login</code>	E-mail e senha do CLIENTE	HTTP 200, token JWT, perfil CLIENTE
T03	Criar pedido aprovado	POST <code>/pedidos/</code>	Unidade, canal APP, PIX e itens	HTTP 201, EM_PREPARO, pagamento APROVADO
T04	Listar pedidos por canal	GET <code>/pedidos/?canal_pedido=APP</code>	<code>canal_pedido=APP</code>	HTTP 200, lista filtrada por canal
T05	Consultar saldo de fidelidade	GET <code>/fidelidade/saldo</code>	Token do cliente	HTTP 200, pontos acum./resg./disp.
T06	Resgatar pontos	POST <code>/fidelidade/resgatar</code>	Quantidade de pontos	HTTP 200, saldo atualizado
T07	Consultar auditoria como ADMIN	GET <code>/auditoria/</code>	Filtros opcionais	HTTP 200, lista de logs
T08	Login com senha inválida	POST <code>/auth/login</code>	Senha incorreta	HTTP 401, credenciais inválidas

ID	Cenário	Endpoint	Entrada	Saída esperada
T09	Acesso à auditoria sem token	GET /auditoria/	Sem Authorization	HTTP 401, erro de autenticação
T10	Cliente acessa auditoria	GET /auditoria/	Token CLIENTE	HTTP 403, erro de permissão
T11	Resgatar pontos acima do saldo	POST /fidelidade/resgatar	Quantidade > saldo	HTTP 409, saldo insuficiente
T12	Criar pedido produto inexistente	POST /pedidos/	Produto inexistente	HTTP 404, produto não encontrado
T13	Consultar pedido por ID	GET /pedidos/{id}	ID salvo em pedido_id	HTTP 200, dados do pedido e pagamento
T14	Criar pedido recusado	POST /pedidos/	Pagamento MOCK_RECUSADO	HTTP 201, PAGAMENTO_RECUSADO, RECUSADO
T15	Criar unidade teste	POST /unidades/	Nome, cidade, estado, endereço	HTTP 201, unidade criada e ativa
T16	Soft-delete de unidade	DELETE /unidades/{id}	ID da unidade teste	HTTP 204, unidade inativa
T17	Listar unidades ativas	GET /unidades/	Sem body	HTTP 200, apenas unidades ativas
T18	Atualizar status: EM_PREPARO → PRONTO	PATCH /pedidos/{id}/status	novo_status: PRONTO	HTTP 200, status_atual PRONTO
T19	Transição de status inválida	PATCH /pedidos/{id}/status	novo_status: EM_PREPARO (de PRONTO)	HTTP 422, TRANSICAO_INVALIDA

Tabela 20 — Cenários de Teste (19 casos, 52/52 asserções)

8.3 Cobertura dos Testes

Categoria	Cobertura
Autenticação	Login válido de ADMIN e CLIENTE; login inválido
Autorização	Acesso bloqueado sem token e com perfil inadequado
Fluxo principal	Pedido aprovado e pedido recusado
Pagamento mock	Cenário aprovado com PIX e recusado com MOCK_RECUSADO
Pedido	Criação, consulta por ID, filtro por canal e atualização de status pela cozinha
Fidelidade	Consulta de saldo, resgate e erro de saldo insuficiente
Auditoria	Consulta como ADMIN e bloqueio para CLIENTE
Unidades	Criação, soft-delete e listagem de unidades ativas
Regras de negócio	Produto inexistente, saldo insuficiente, acesso não autorizado e transição inválida de status

8.4 Validação Complementar

Além dos testes Postman, foi realizada validação local de sintaxe e importação dos arquivos principais da aplicação com o comando `python -m compileall app main.py seed.py`. A execução foi concluída sem erros, indicando que os principais módulos Python estão sintaticamente corretos. Também foi validado manualmente o acesso ao OpenAPI JSON com retorno HTTP 200.

9 CONCLUSÃO

O projeto desenvolvido entrega uma API REST funcional para gestão de pedidos da rede Raízes do Nordeste. A solução cobre o fluxo principal de uma operação de restaurante: autenticação de usuário, consulta de unidades e produtos, controle de estoque, criação de pedido, processamento de pagamento mock, atualização de status, acúmulo de pontos de fidelidade e auditoria.

A escolha do FastAPI permitiu construir uma API moderna, com documentação automática via Swagger/OpenAPI e validação de dados com Pydantic. O uso do SQLAlchemy facilitou a modelagem das entidades e o acesso ao banco SQLite. A autenticação com JWT e o controle de permissões por perfil fornecem uma base inicial de segurança adequada ao escopo do projeto.

O pagamento mock foi implementado de forma determinística, permitindo testar fluxos aprovados e recusados de maneira previsível. Isso facilita a validação via Postman e evita instabilidade nos testes, garantindo que os cenários documentados possam ser reproduzidos de forma consistente.

A modelagem apresentada conecta o DER, as classes de domínio, os casos de uso e o diagrama de sequência ao fluxo crítico entregue. O pedido se relaciona com usuário, unidade, itens, pagamento, estoque, fidelidade e auditoria, refletindo diretamente os principais endpoints implementados.

O fluxo de cozinha foi implementado via endpoint `PATCH /pedidos/{pedido_id}/status`, permitindo que ADMIN e GERENTE atualizem o status dos pedidos aprovados seguindo transições válidas: `EM_PREPARO` → `PRONTO` → `ENTREGUE` ou `EM_PREPARO` → `CANCELADO`. Transições inválidas retornam HTTP 422 com mensagem padronizada, e todas as atualizações são registradas em auditoria.

Alguns itens do estudo de caso foram mantidos como proposta, como promoções e campanhas. Essa escolha foi feita para manter o escopo do MVP viável e priorizar o fluxo principal executável, testável e persistente.

Com isso, o projeto demonstra domínio dos principais elementos de uma aplicação back-end: modelagem de dados, criação de endpoints REST, autenticação, autorização, persistência, validação de entrada, regras de negócio, tratamento de erros, controle de estoque, fidelização e auditoria. A solução atende ao objetivo acadêmico proposto e apresenta uma base técnica organizada, funcional e testável.

REFERÊNCIAS

BRASIL. **Lei n. 13.709, de 14 de agosto de 2018.** Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

FASTAPI. **FastAPI Documentation.** Disponível em: <https://fastapi.tiangolo.com>. Acesso em: 5 jun. 2026.

FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures.** University of California, Irvine, 2000.

FOWLER, Martin. **Patterns of Enterprise Application Architecture.** Boston: Addison-Wesley, 2002.

PASSLIB. **Passlib Documentation.** Disponível em: <https://passlib.readthedocs.io>. Acesso em: 5 jun. 2026.

PYDANTIC. **Pydantic Documentation.** Disponível em: <https://docs.pydantic.dev>. Acesso em: 5 jun. 2026.

PYTHON-JOSE. **Python-JOSE Documentation.** Disponível em: <https://python-jose.readthedocs.io>. Acesso em: 5 jun. 2026.

RICHARDSON, Leonard; RUBY, Sam. **RESTful Web Services.** Sebastopol: O'Reilly Media, 2007.

SQLALCHEMY. **SQLAlchemy Documentation.** Disponível em: <https://docs.sqlalchemy.org>. Acesso em: 5 jun. 2026.